Modifications made - items 1 to 8

FeedXchange backend (`C:\FeedXchange\FeedXchange`, branch main) and frontend (`C:\FeedXchange\ReactProject_Dev`, branch dev). Prepared 14 November 2026. Nothing has been committed or pushed in either repository.

1. Summary

#	Item	Result
1	Child businesses: rules and visibility	Done. Server rules with clear messages; parent sees children's contracts, invoices, payments and notification history; child screen wired to the real APIs.
2	Business contacts not saved	Done. Root cause: the profile save ignored the contacts list. Primary and additional contacts are now stored and returned.
3	Saving a seller invoice marks the truck Loaded	Done. Invoice with contractDispatchId marks that dispatch Loaded and links it; no trip question; manual Loaded without an invoice is refused; Update Truck Status opens Create Seller Invoice.
4	Dispatched quantity only after a seller invoice	Done. One calculator for arranged / dispatched / pending and status; existing data corrected by a script (run on the local dev database).
5	Contract notifications (a to e)	Done. Contacts before save, one template per role, preview without a saved contract, plan stored with the contract and sent on approval, TRUCK_LOADED sent when the invoice marks Loaded.
6	Document downloads broken	Done as chosen: records kept, screen shows 'File not available'; the API reports which files are missing; Documents folder is created if absent.
7	Remove profile sub-menu items	Done. 13 pages removed from the page data and seed scripts, with their saved role / user access; permissions kept.
8	Seller and buyer city from the API	Done (backend + database). City on business lists; sellerId / sellerCity / buyerId / buyerCity on the contract, invoice, payment, truck and notification responses; city saved with each contract.

Your decisions that shaped the work

- Relationship type: accept both **Child** and **Group** (existing demo link uses Group).
- Contract status when trucks are only arranged: **PartiallyDispatched once any truck is arranged**; Dispatched only when the invoiced quantity reaches the contract quantity.
- TRUCK_LOADED: **send automatically** when the invoice marks the dispatch Loaded.
- Missing documents (profile ids 2-7): **keep the records** and show 'File not available'.
- Primary contact: stored as a ProfileContact of type 'Primary'. Stored notification plan: contacts are looked up again at approval.
- Database: scripts were run on the local dev database only (server sathesh-laptop, FeedXchangeDB). No other database was touched.
- Frontend: your note said several screens were already done; they were not in this working copy, so they were built here.

2. Item 1 - Child businesses: rules and visibility

Accepted relationShipType values

The column was free text. The server now accepts **Child** and **Group** (case-insensitive, stored in canonical spelling) and rejects anything else with: *Relationship type "X" is not valid. Allowed values: Child, Group.* The frontend sends 'Child'.

Rules enforced on LinkChildBusinessProfiles

Rule	Message returned (business name shown)
A business cannot be its own child	"X" cannot be linked as its own child.
A child can have only one parent	"X" is already a child of another business and can have only one parent. De-link it from its current parent first.
A business with children cannot become a child	"X" has child businesses of its own and cannot become a child.

Rule	Message returned (business name shown)
Already linked to this parent	"X" is already a child of this business.
Nothing selected / unknown profile	Select at least one business to link as a child. / Child profile(s) not found: ...

- Every selected business is checked first; if any one is blocked **nothing is linked**. The message lists the first reasons and '(N more blocked.)' so it stays short enough for the screen to show.
- Linking a pair that was de-linked earlier switches the old (inactive) row back on instead of failing on the unique index.

Visibility

A new `VisibleProfileIds` on the current-user service holds the caller's own profile plus its active Child/Group children. It is filled once per request in `CurrentUserMiddleware` and used wherever business users are limited to their own contracts: **contract** APIs (ContractRepository), **seller invoice** APIs (SellerInvoiceRepository), **payment** APIs (PaymentAdviceRepository: pair check, receipt access) and **notification** access/history (NotificationService). The child never sees the parent's data. Platform staff are unchanged. The system has no separate report API - the reports read these contract / invoice / payment APIs, so they follow the same rule.

Frontend

`LinkedBusinessTab.tsx` used mock data. It now calls GetAllChildBusinessProfiles, LinkChildBusinessProfiles ({parentProfileId, relationShipType:'Child', createdBy, createdOn, childProfileIds}) and DelinkChildBusinessProfile, shows the server's message when a rule blocks a link, and offers only businesses the server could accept. New RTK Query endpoints in `businessProfilesApi.ts`.

3. Item 2 - Business contacts not saved

Root cause. The screen sent a `contacts` list with the profile save, but `UpdateBusinessProfileDto` had no such property, so the list was dropped silently. The primary contact's name was never sent at all (only kept in the form). For SL International (profile 83) the contacts were therefore never stored; they must be entered and saved once again after this release.

- **Backend:** new `UpdateBusinessProfileContactDto` and optional `Contacts` on UpdateBusinessProfileDto. `SyncProfileContactsAsync` updates a contact in place when it carries a ContactId (so its notification preferences are kept), adds new ones, removes the ones left out, ignores blank rows, and avoids copies when an older screen sends no id. When `Contacts` is left out the stored contacts are not touched. GetBusinessProfileById already returned contacts.
- **Frontend:** the Contacts & Addresses step now sends the primary contact (type 'Primary', name from Primary Contact, mobile / alternate / email from the profile fields) and the additional contacts, with their ids. Other wizard steps do not send contacts, so they cannot wipe them. On reload the 'Primary' contact fills the Primary section and the rest fill Additional Contacts (before, the first contact appeared in both).
- Because contacts now exist, the profile's Notifications tab lists their names and numbers.

4. Items 3 and 4 - Seller invoice marks Loaded; dispatched quantity

Item 3

- **SaveInvoices with contractDispatchId** marks that dispatch Loaded (whole re-assignment chain, as before), links the invoice to it, and **never returns requiresTripSelection**: the named dispatch decides the truck. If the typed truck number does not match the dispatch's truck, or the dispatch is not on that contract / is cancelled, a clear error is returned and nothing is saved.
- **UpdateContractTruckStatus** (public API) refuses Loaded unless a live seller invoice exists for that dispatch: *A truck can be marked Loaded only after its seller invoice is saved...* The invoice save itself still performs the Loaded transition internally.
- The old rule 'only a Loaded trip can be selected when two or more are loaded' no longer applies when the invoice names its dispatch (it would have blocked the new flow).
- **Frontend:** in Update Truck Status, picking Loaded shows a **Create Invoice** button that opens Create Seller Invoice with contract, truck, dispatch and quantity filled in (contract and truck read-only). The invoice is sent with contractDispatchId; after saving you return to the contract's trucks. The 'Truck loaded' prompt was removed from this screen (item 5e).

Item 4 - one rule, one place

New `ContractQuantityCalculator` (Infrastructure). Contract quantity columns are written by this code only, so the contract list, GetAllOpenAndPendingContracts, the full-details summary, pending / arranged quantities and the status all read the corrected stored values.

Value	Rule
Dispatched	Sum of the quantity of the contract's live (not cancelled) seller invoices linked to a truck dispatch, capped at the contract maximum. A Loaded truck without an invoice does NOT count.
Arranged	Quantity of live trucks that have no live invoice: Assigned trucks and Loaded trucks waiting for an invoice.
Pending	Total - arranged - dispatched (never below 0).
Status	Dispatched when dispatched >= total; PartiallyDispatched while anything is arranged or dispatched. Only Open / PartiallyDispatched / Dispatched contracts change; Pending, Closed, Rejected, Deleted keep their status.
When it runs	After a truck status change (Loaded / Cancelled), after saving, editing or cancelling an invoice, and the status line of the three 'arrange truck' paths (add trucks, edit quantity, re-assign) no longer shows Dispatched merely because everything is arranged.

Existing data

`DatabaseScripts/FixDispatchedQuantityFromInvoices.sql` recomputes every contract from its trucks and invoices, prints a before / after list, keeps the old values in `dbo.ContractQuantityFixBackup`, and is safe to run twice. Run on the local dev database it corrected 9 contracts (e.g. 2026-6 and 2026-11 went from Dispatched back to PartiallyDispatched). **Note:** invoices that are not linked to a truck dispatch (older invoices) do not count as dispatched under this rule, so some contracts show dispatched 0 until their invoices are linked.

5. Item 5 - Contract notifications

Part	Backend	Frontend
a) Contacts before save	GET /api/Notifications/recipients?profileId=ID&eventCode=CONTRACT_CREATED works with no contract (also CONTRACT_UPDATED). Returns displayName, role, mobile, email, hasAppLogin. Optional **role=SELLER|BUYER** sets the role shown (left out = SELECTED). Business users may ask only for themselves, their children and businesses they already trade with.	NewContract shows each party's contacts (names, numbers, App login badge) as soon as the seller / buyer is picked, or a clear 'no contacts' note.
b) One template per role	The send / preview request accepts **roleTemplates** [{role, channelCode, templateId}]; it is used for every contact of that role and channel unless a recipient's own channel names a template.	One template per role and channel, applied to all that party's contacts (NotifyChoicePicker).
c) Preview before saving	New **POST /api/Notifications/preview-draft**: eventCode, items [{role, channelCode, templateId}], draft {sellerId, buyerId, productId, quantityMT, ratePerMT, gstPercentage, paymentTerms, due days, contractDate}. Returns one rendered message per role and channel.	A preview (eye) icon beside each template shows the message filled with the contract being entered.
d) Saved as Pending	Plan stored with the contract (tables ContractNotificationPlans / Items). SaveContract and UpdateContract accept **notificationPlan**. When the contract is approved - Contract Status Approve, or an edit that approves it, or saved Approved - the plan is queued once through the notification outbox for each party, to that party's **current** contacts that can be reached (a missing address only drops that channel). GET /api/Notifications/contract-plan?contractNumber= returns it.	Choices are saved with the contract and pre-filled when editing. The Contract Status screen tells the user the stored plan will be used and no longer asks again (ticking its box still sends the separate 'approved' message).
e) Truck loaded	When saving a seller invoice marks a dispatch Loaded, the buyer is sent TRUCK_LOADED automatically (default channels of the event rules for the buyer, default template), once per dispatch (idempotency key). A failure never undoes the saved invoice.	The manual 'Truck loaded' prompt is gone.

- A Pending contract always sends the 'contract created' message on approval, even if 'updated' was chosen on the form (it was never announced).
- The Notifications section is now before Approval (numbered 4 and 5), and the post-save notification dialog on the contract form was removed because the plan is stored and sent automatically.

- New contracts are now notified by default (Pending ones when approved) unless back-dated; edits never notify unless ticked.
- Sending is queued through the existing notification outbox, so the notification workers must be running (as they already must for every notification).

6. Item 6 - Document downloads

- **Cause for ids 2-7:** the seed records point to /documents/demo/*.pdf, a folder that does not exist on the server. ID 83 works because a real upload wrote to the folder the download URL reads.
- **Backend:** GetBusinessProfileById adds `fileAvailable` to each document (true when the file is on disk under Documents). Program.cs creates the Documents folder when missing (the static-file provider needs it to start).
- **Frontend:** the Documents table shows 'File not available' instead of a download link that would only return 404, and shared document lists skip those links.
- **Uploads:** files are written to <working folder>/Documents/... and served from <content root>/Documents at /documents. These are the same folder with the default hosting; no change was made to the upload code (a change to the working folder made the test suite delete sample images, so it was removed).
- The records were kept as you chose. To make them downloadable, upload the files again from Edit, or tell me to remove the records.

6a. Item 7 - Profile sub-menu items removed from the menu and access setup

- **Removed (13 pages):** Business Owners: Profile, Contacts & Addresses, Documents, Bank Details, Profile Settings (/business-owners/...); Transporters: the same five (/transporters/...); Promoters: Profile, Documents, Profile Settings (/promoters/...). The list pages and the Detail screens are untouched.
- **Notification Preferences sub-items:** there were none in the page table (the notification-preferences page is a separate top-level page), so there was nothing to remove for them.
- **Script:** `DatabaseScripts/RemoveProfileWizardMenuPages.sql` deletes the pages, their page-action rows and the role / user access saved for them (no orphan entries in Assign Role / Assign User), in one transaction. Safe to run twice. On the local dev database it removed 13 pages, 39 page actions and 18 role access rows.
- **Seed scripts:** the same rows were taken out of `AddModulesPagesTables.sql` and `SeedModulesSubModulesPages.sql` (13 page rows and 39 page-action rows) so re-running them does not bring the items back.
- **Permissions kept:** profiles.business.manage, profiles.transporter.manage, profiles.promoter.manage and notification-preferences.view are not touched; the wizard is still opened from New and Edit on each list.

6b. Item 8 - Seller and buyer city returned by the API

How the city is decided. A business' city = its primary address' city, else its first address' city (inactive one-time contract addresses are ignored). A contract stores the city it was made with: the loading address' city for the seller and the delivery address' city for the buyer, or the business' city when no contract address is set. Old contracts therefore keep their city when a business later changes its address.

Database (DatabaseScripts/AddPartyCityToContractsAndLists.sql)

- New columns `Contracts.SellerCity` and `Contracts.BuyerCity`; every existing contract is filled by the same rule (run on dev: 12 of 12 filled).
- New function `dbo.fn_ProfileCity` (primary, else first active address).
- Procedures changed (columns only added at the end): GetBusinessProfileSummary and GetAllChildBusinesses, GetTransporterProfileSummary, GetPromoterProfileSummary return **City, StateName** (the business summary used a plain primary-address join that could repeat a business; it now returns one row per business and falls back to the first address); GetAllOpenAndPendingContracts returns SellerId, SellerCity, BuyerId, BuyerCity; GetContractTruckChain / Overview and GetScheduledDispatches return the stored cities.

API responses (city, and the id where it was missing, next to every seller / buyer name)

Area	Responses
Business lists	GetBusinessProfileSummary, GetAllChildBusinessProfiles, transporter and promoter summaries: **city**, **stateName**.

Area	Responses
Contracts	GetAllContracts, GetAllContractsByFilters, GetContractByContractNumber, contracts for transporter, GetAllOpenAndPendingContracts: sellerId / sellerCity / buyerId / buyerCity from the city saved with the contract. Contract full details: **city** inside the seller and buyer blocks.
Seller invoices	Contract lookup and invoice contracts, saved invoices (Edit Seller Invoices): sellerCity / buyerCity. Seller and buyer lookups: **{ id, name, city }**.
Payments	Seller / buyer lookups **{ id, name, city }**; advance contracts, receipts, refunds and invoices to clear: sellerId / sellerCity / buyerId / buyerCity (the contract's saved city when the record has a contract, otherwise the business' city).
Contract trucks	Truck chain and chain overview (legs, participants, first seller / final buyer), scheduled dispatches: sellerCity / buyerCity.
Notification history	History list and delivery detail: sellerId / sellerCity / buyerId / buyerCity (the delivery detail now also fills contract number and names); history sellers / buyers **{ id, name, city }**; history contracts: ids and cities.
Contracts dashboard	The dashboard has no endpoint of its own: top buyers and sellers are built from the contract list, which now carries the ids and cities.

Saving

When a contract is created, duplicated or edited, the seller and buyer city are worked out and saved with it (`PartyCities.SetContractCitiesAsync`); an edit that changes the party or an address updates the stored city. The frontend was not changed for this item, as you said it will read the new fields itself; it can now drop the per-business profile requests and name matching.

7. Files changed

Backend (C:\FeedXchange\FeedXchange)

File(s)	Change
Authorization/CurrentUser.cs, ICurrentUser.cs	VisibleProfileIds (self + active children), filled by the middleware
Domain/Entities/ProfileRelationshipTypes.cs (new)	Allowed relationship types: Child, Group
BusinessProfileRepository.cs	Link rules and messages; contacts sync; de-linked pair re-activation
UpdateBusinessProfileDto.cs, UpdateBusinessProfileContactDto.cs (new)	Contacts on the profile save
BusinessProfilesController.cs, GetProfileDocumentDto.cs, Program.cs	fileAvailable flag; Documents folder created
ContractRepository.cs, SellerInvoiceRepository.cs, PaymentAdviceRepository.cs, NotificationService.cs	Parent sees children's data
ContractQuantityCalculator.cs (new)	Arranged / dispatched / pending / status rule
ContractTrucksRepository.cs, ContractTrucksService.cs, IContractTrucksRepository.cs	Loaded needs an invoice; calculator used; status after arranging
SellerInvoiceRepository.cs, SellerInvoiceRepository.Trips.cs, SellerInvoiceService.cs	Named dispatch; recalculation on save / edit / cancel; TRUCK_LOADED
NotificationService.cs, MessageData.cs, NotificationsController.cs, NotificationDtos.cs, NotificationInterfaces.cs	Profile-only recipients, role templates, draft preview, plan endpoint
ContractNotificationPlanService.cs, IContractNotificationPlanService.cs, ContractNotificationPlanEntities.cs, ContractNotificationPlanConfigurations.cs, FeedXchangeDBContext.cs, NotificationServiceCollectionExtensions.cs (all new/edited)	Stored plan, send on approval, TRUCK_LOADED
SaveContractDto.cs, ContractService.cs	notificationPlan on save / update; send on approval
DatabaseScripts/AddContractNotificationPlans.sql, FixDispatchedQuantityFromInvoices.sql (new)	Two new tables; data correction
Tests: DispatchedQuantityAndProfileRulesTests.cs, ContractFormNotificationTests.cs (new)	Cover items 1-5 and 8 against a real SQL Server schema
DatabaseScripts/RemoveProfileWizardMenuPages.sql (new), AddModulesPagesTables.sql, SeedModulesSubModulesPages.sql	Item 7: pages and access removed; seeds cleaned
DatabaseScripts/AddPartyCityToContractsAndLists.sql (new)	Item 8: city columns, function, backfill, 8 procedures
PartyCities.cs (new), Contract.cs, ContractConfiguration.cs, ContractRepository.cs (+ FullDetails), ContractService.cs	Item 8: stored contract city; contract responses
ContractAggregateData.cs, GetAllContracts*Dto.cs, GetContractDto.cs, ContractDetailsDto.cs, ContractDetailsSPVM.cs, ContractFullDetailsDto.cs	Item 8: city / id fields on contract responses
SellerInvoiceDtos.cs, SellerInvoiceRepository.cs, PaymentAdviceDtos.cs, PaymentAdviceRepository.cs	Item 8: invoice and payment responses and lookups
GetContractTruckChain*.cs, ContractTruckChainOverviewDto.cs, ScheduledDispatchDetailsSPVM.cs, GetScheduledDispatchesDto.cs, ContractTrucksRepository.cs	Item 8: truck responses
NotificationDtos.cs, NotificationService.cs	Item 8: history responses
BusinessProfileSummarySPVM / GetBusinessProfileSummaryDto, Transporter and Promoter summary SPVM / DTOs	Item 8: city and stateName on lists

Frontend (C:\FeedXchange\ReactProject_Dev\src\...)

File(s)	Change
core/store/businessProfilesApi.ts, userProfilesCommonApi.ts	Child endpoints; contactId on contacts; fileAvailable

File(s)	Change
BusinessNew/BusinessOwnerWizardContext.tsx, ContactsAddresses.tsx, businessOwnerWizard.utils.ts, useSaveContactsAddressesStep.ts	Primary + additional contacts saved and reloaded correctly
BusinessView/LinkedBusinessTab.tsx	Real link / de-link / list of child businesses
BusinessView/BankDetailsTab.tsx, businessOwnerDetail.data.ts, BusinessOwnerDetail.scss	'File not available' for missing documents
update-truck-status/UpdateTruckStatus.tsx	Loaded opens Create Seller Invoice; no truck loaded prompt
payments/seller-invoice/SellerInvoice.tsx, InvoiceBlock.tsx, sellerInvoice.data.ts	Pre-filled contract, truck, dispatch; sends contractDispatchId
core/store/notificationsApi.ts, contractsApi.ts	preview-draft, contract-plan, role, notificationPlan
shared/notifications/NotifyChoicePicker.tsx	Contacts per party, one template per role, preview icon
contracts/NewContract.tsx, contract-status/ContractchangeStatus.tsx	Notifications before Approval; plan saved with the contract; no second prompt on approval

8. Deployment and the database

Step	What
1	On each database (Dev, then Prod) run `AddContractNotificationPlans.sql` **before** the new API is deployed (creates ContractNotificationPlans and ContractNotificationPlanItems).
2	Deploy the API and the frontend.
3	Run `FixDispatchedQuantityFromInvoices.sql` once on each database; review the printed before / after list. The old values stay in ContractQuantityFixBackup.
4	Re-enter and save the contacts of businesses that lost them (for example SL International, profile 83): Edit, Contacts & Addresses, Save.
5	Re-upload the missing documents of the sample businesses (ids 2-7), or ask for the records to be removed.
6	Item 7: run `RemoveProfileWizardMenuPages.sql` on each database (any time).
7	Item 8: run `AddPartyCityToContractsAndLists.sql` on each database **before** deploying the API - the API reads the new columns and the changed procedure results.

Already applied on the local dev database (sathesh-laptop / FeedXchangeDB): both scripts. Dev and Prod servers were not touched.

9. Testing

- Backend: whole solution builds; **80 of 80 tests pass** with every database-backed suite switched on (76 existing + 4 new), using throw-away scratch databases that were dropped afterwards.
- The item 8 test proves the city rules (primary, else first address; inactive ignored; the contract address wins; another business' address is never used) and that the lookups and the contract lookup return the saved city. New tests prove: dispatched counts only after the invoice (arranged 10 / dispatched 0 -> after invoice 5 / 4 -> 0 / 9 -> cancelled back to 5 / 4); a named dispatch is marked Loaded with no trip question; a mismatching truck is refused; all child-business rules and messages; parent-versus-child visibility; contacts saved, updated in place, removed, and left alone when not sent; recipients with only a profile id; one message per role in the draft preview; one template for every contact of a role; the plan stored, sent once on approval to the current contacts (a contact added later is included; a contact with no email gets WhatsApp only); TRUCK_LOADED queued and resolved to a real send.
- Items 7 and 8: both scripts were run on the local dev database and the changed procedures were executed there (business, child, transporter, promoter, open / pending contracts, scheduled dispatches, truck chain and overview). Frontend: TypeScript check is clean and a production build succeeds. The new screens were **not** clicked through in a browser.
- The API starts with the new routes registered. I did not log in to exercise authenticated endpoints end to end.
- Two older opt-in notification tests (ContractCreatedEndToEndTests and StageNotificationsEndToEndTests) fail with 'Invalid column name IsMandatory' because their setup never applies the preferences script; they fail the same way on the original code and are unrelated.

10. Things to know

- A business that already is a child can later be given children of its own; the rules you listed do not forbid it, so it is not blocked.
- A parent user sees its children's contracts, invoices and payments through the same filter the child's own users use, so a parent can also act on them where the screen allows (edit, invoice, receipt).
- The preview is a sibling route (/api/Notifications/preview-draft) rather than a change to /preview, because /preview requires recipients and a saved contract. The draft preview picks the with / without quality wording from whether the product has quality features.
- Invoices not linked to a truck dispatch are not counted as dispatched (rule 4); the contract view's Invoiced bar shows the dispatched (invoiced) quantity as you described.
- Removing a contact from the profile also removes that contact's notification preferences (they hang off the contact).
- Items 7 and 8 need no frontend change in this release (the menu items were already removed there; the frontend will read the new city fields).
- Nothing was committed or pushed. Review the changes in both repositories and commit when ready.